Step 3 — Rendering: una domanda, il prompt nativo, i documenti lunghi

Obiettivo

Trasformare ogni riga del mix in un esempio tokenizzato **identico a quello che il modello vedrà in produzione**: prompt nativo, una sola domanda (quella scelta allo Step 1 e scritta in `finetune/question.json`), target sulle due lettere. In più: decidere cosa fare dei documenti più lunghi del budget di token, sia in training sia in inferenza (chunking alla MiniCheck/FactCG).

Input e output

Input	`mix_fc_*/train.jsonl`, `dev_select`, `dev_confirm`, `finetune/question.json`
Output	`data/finetune/rendered/<modello>/<mix>/{train,dev_select,de`
Codice	`finetune/render.py` (**da scrivere**; lo stesso del piano generale, con la sola modalità S1)

1. Il prompt (esempio con `support_simple`)

Si importano le funzioni dei benchmark, non si riscrive nulla: `safe_data`, `Scorer.render`, `Scorer.suffix` da winnow-12b-bf16/benchmark.py; `llm2jev.prompt.render`, `find_labels` per JPT (jpt-9b/benchmark.py). Il testo esatto dei due prompt è in ../fine_tuning/03_rendering_formato.md §2–3 e in ../prompt_formats.md.

	Winnow-12B	JPT-9B
Stato	JSON compatto `{"document":...,"claim":...}`	righe `document: ...` / `claim: ...`
Opzioni	A = falso, B = vero	A = sì, B = no
Token letti	`A` 236776, `B` 236799	`" A"` 357, `" B"` 417
Temperatura	1	1,087
Target per label 1	`[0, 1]`	`[1, 0]`
Target per label 0	`[1, 0]`	`[0, 1]`

Con il compito unico **un esempio per riga** (modalità S1 del piano generale): niente seconda e terza domanda, niente replay. Un mix da 60k righe = 60k esempi.

2. Formato di un esempio renderizzato

```
{"id": "minicheck:train:c2d:0", "model": "jpt-9b",
 "input_ids": ["<token del prompt>"], "answer_pos": "<indice dell'ultimo token>",
 "letter_ids": [357, 417], "target": [1.0, 0.0],
 "stage": 1, "source": "minicheck_c2d", "neg_type": null,
 "chunk": null, "teacher_p": 0.83}
```

- `neg_type`: `contradicted` / `not_established` / `null` (positivi o sconosciuto): serve per analisi e per controllare la quota di negativi "non stabilito".
- `teacher_p`: P(supportato) del modello di partenza, solo se si usa l'ancoraggio P-KL (02c).
- `chunk`: indice del chunk se il documento è stato spezzato (§4).

3. Budget di token

Fase	Budget del prompt	Perché
Pilota (JPT, QLoRA sul portatile)	4.096	memoria

Fase	Budget del prompt	Perché
Corsa piena (L40S)	8.192	documenti D2C, SummEdits, trascrizioni
Inferenza	fino al contesto nativo, ma con chunking oltre una soglia fissata sul dev (§4)	qualità e costo

4. Documenti lunghi: chunking, come MiniCheck e FactCG

In inferenza

```
documento → frasi (NLTK) → chunk consecutivi fino a C token (sui confini di frase)
per ogni chunk k:  p_k = P(supportato | chunk_k, claim)
score = max_k p_k
```

È la regola di MiniCheck e FactCG (chunk di 400–550 token per gli encoder). Per un LLM i chunk possono essere molto più grandi: Bespoke-MiniCheck-7B usa ~32k token. Il valore di C si sceglie **solo su `dev_select`** (le sue righe lunghe), con griglia C ∈ {2k, 4k, 8k, intero}; `test_long` si guarda solo alla fine.

Limite noto della regola `max`: se l'evidenza per due parti del claim è in due chunk diversi, nessun chunk da solo supporta il claim e il risultato è "falso". Per questo i chunk di un LLM devono essere grandi: meno chunk, meno catene spezzate.

Opzione (da Bespoke-MiniCheck-7B): spezzare anche il **claim** in frasi e usare `score = min_frase max_chunk p`. Utile per risposte lunghe (LFQA, ExpertQA); da provare in K2 (07).

In training

Un documento oltre il budget va gestito diversamente secondo l'etichetta:

Etichetta	Si può spezzare?	Perché
0 (non supportato)	**sì**: ogni chunk diventa un esempio negativo	se il documento intero non supporta il claim, nessuna sua parte lo supporta
1 (supportato)	**no**, salvo evidenza localizzata	un chunk potrebbe non contenere l'evidenza → etichetta sbagliata. Si usa il documento intero (budget più alto), il chunk che contiene le frasi gold (FEVER, HoVer, C2D: si sa dove sono), oppure si scarta la riga

Contare per fonte le righe scartate o spezzate; oltre l'1% va segnalato.

5. Traccia encoder (controllo E1 e studente della distillazione)

Per DeBERTa-v3-large si usa il template di FactCG-DBT, una sequenza unica (non la coppia `[CLS] doc [SEP] claim [SEP]`):

```
{chunk}

Choose your answer: based on the paragraph above can we conclude that "{claim}"?

OPTIONS:
- Yes
- No
I think the answer is
```

Chunk da ~550 token, massimo 2.048 token in training, target `[P(no), P(sì)]` sulla testa `Linear(1024→2)`. Il codice di training di FactCG (derenlei/FactCG, `factcg/dataloader.py`) si può riusare quasi intero; in `FactCG/` nella repo c'è già il codice di inferenza e chunking (FactCG/chunking.py).

6. Test obbligatori di `render.py`

Test	Cosa verifica
Identità con il benchmark	20 righe di 5 fonti: `input_ids` identici a quelli dei benchmark con la domanda scelta
Lettere	id ricavati dal tokenizer = 236776/236799 (Winnow), 357/417 (JPT)
Target	Winnow label 1 → B; JPT label 1 → A
Chunking	un documento lungo produce chunk sui confini di frase, ciascuno entro il budget; i positivi non vengono spezzati

Criteri per passare allo step successivo

- Test di §6 superati.
- Statistiche: esempi per stadio e fonte, lunghezza p50/p95 in token, righe spezzate o scartate per fonte.